CAD for VLSI Design
Programming Assignment 1

Benchmark Translator Project Report

Advisor: Yu-Guang Chen

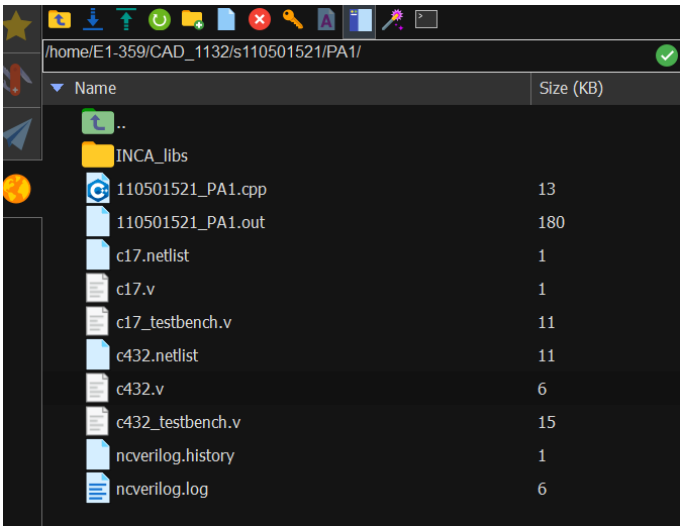
Student Info: 電機 4A

Student ID: 110501521

Student Name: 徐松廷

1. How to compile and execute your program

a. Step 1:登入工作站後，將 code.cpp、.testbench 與.netlist file 上傳到當前的工作目錄



b. Step 2:依序執行以下指令來編譯定執行程式，並產生.v file:

```
g++ -std=c++11 110501521_PA1.cpp -o 110501521_PA1.out
./110501521_PA1.out c17.netlist c17.v
```

c. Step 3: 依序執行以下指令，以 NC verilog 依照.testbench 做模擬並確認.v file 是否可合成:

```
source /usr/cad/cadence/CIC/incisiv.cshrc
source /usr/cad/synopsys/CIC/verdi.cshrc
```

d. Step 4:執行結果如下:

```
[s110501521@ncuee ~/PA1]$ g++ -std=c++11 110501521_PA1.cpp -o 110501521_PA1.out
[s110501521@ncuee ~/PA1]$ ./110501521_PA1.out c17.netlist c17.v
writeback successfully
[s110501521@ncuee ~/PA1]$  source /usr/cad/cadence/CIC/incisiv.cshrc
set INCISIV version: INCISIVE_15.20.084 (default)
[s110501521@ncuee ~/PA1]$  source /usr/cad/synopsys/CIC/verdi.cshrc
set verdi version: 2019.06/ (default)
[s110501521@ncuee ~/PA1]$ ./110501521_PA1.out c17.netlist c17.v
writeback successfully
[s110501521@ncuee ~/PA1]$ ncverilog +access+r c17_testbench.v c17.v
```

```
ncverilog(64): 15.20-s084: (c) Copyright 1995-2020 Cadence Design Systems, Inc.
file: c432_testbench.v
        module worklib.c432_tb:v
                errors: 0, warnings: 0
file: c432.v
        module worklib.c432:v
                errors: 0, warnings: 0
                Caching library 'worklib' ....... Done
        Elaborating the design hierarchy:
        Building instance overlay tables: ..................... Done
        Generating native compiled code:
                worklib.c432_tb:v <0x4b9884ca>
                        streams:  42, words: 107254
        Building instance specific data structures.
        Loading native compiled code:     ..................... Done
        Design hierarchy summary:
                                Instances  Unique
                Modules:                2       2
                Primitives:           160       5
                Registers:             37      37
                Scalar wires:          36       -
                Initial blocks:         1       1
                Pseudo assignments:    36      36
                Simulation timescale:  1ps
        Writing initial simulation snapshot: worklib.c432_tb:v
Loading snapshot worklib.c432_tb:v ..................... Done
ncsim> source /usr/cad/cadence/INCISIV/INCISIVE_15.20.084/tools/inca/files/ncsimrc
ncsim> run
 input pattern = 1111100000111110000011111100000111110--> golden value = 1001011
 your answer = 1001011
 input pattern = 1100111111111111111110000000000001100 --> golden value = 1111000
 your answer = 1111000
 input pattern = 0000011111111111111110000000000000000 --> golden value = 1001111
 your answer = 1001111
 input pattern = 0000011111111111111110000000000011111 --> golden value = 1001111
 your answer = 1001111
 input pattern = 1111111111111111111001100000000000000 --> golden value = 1111011
 your answer = 1111011
```

e. Step 4:如果產生笑臉，代表 design compiler 編譯成功

2. The Completion of the Assignment

已完成**所有** document 中的指定功能，詳列如下:

a. 從 netlist 中抓出 module name 與 input + output net，排序後輸出於.v file

```
* Netlist of the module me1

Inst gat1 xor
Inst gat2 xor
Inst gat3 and
Inst gat4 and
Inst gat5 or
Inst outbuffer1 buf
Inst outbuffer2 buf


NET net1 3
PIN inptA, gat1/IN1, gat4/IN1
```

```
`timescale 1ns/1ps
module me1 (N1,N2,N3,N6,N10);

input N1,N2,N3;
output N6,N10;
wire N4,N5,N7,N8,N9;

xor XOR2_1 (N4, N1, N2);
xor XOR2_2 (N5, N4, N3);
and AND2_1 (N7, N4, N3);
and AND2_2 (N8, N1, N2);
or OR2_1 (N9, N7, N8);
buf BUF1_1 (N6, N5);
buf BUF1_2 (N10, N9);

endmodule
```

b. 從 netlist 中分別抓出 output net、input net 與 wire net，排序後輸出於.v file

```
NET net1 3
PIN inptA, gat1/IN1, gat4/IN1
NET net2 3
PIN inptB, gat1/IN2, gat4/IN2
NET net3 3
PIN inptC, gat2/IN2, gat3/IN2
NET net4 3
PIN gat1/OUT1, gat2/IN1, gat3/IN1
NET net5 2
PIN gat2/OUT1, outbuffer1/IN1
NET net6 2
PIN outbuffer1/OUT1, outptSum
NET net7 2
PIN gat3/OUT1, gat5/IN1
NET net8 2
PIN gat4/OUT1, gat5/IN2
NET net9 2
PIN gat5/OUT1, outbuffer2/IN1
NET net10 2
PIN outbuffer2/OUT1, outptCarry
```

```
`timescale 1ns/1ps
module me1 (N1,N2,N3,N6,N10);

input N1,N2,N3;
output N6,N10;
wire N4,N5,N7,N8,N9;

xor XOR2_1 (N4, N1, N2);
xor XOR2_2 (N5, N4, N3);
and AND2_1 (N7, N4, N3);
and AND2_2 (N8, N1, N2);
or OR2_1 (N9, N7, N8);
buf BUF1_1 (N6, N5);
buf BUF1_2 (N10, N9);

endmodule
```

c. 從 netlist 中抓取 gate 的資訊並順利編號，轉成.v 的 gate 形式

```
NET net1 3
PIN inptA, gat1/IN1, gat4/IN1
NET net2 3
PIN inptB, gat1/IN2, gat4/IN2
NET net3 3
PIN inptC, gat2/IN2, gat3/IN2
NET net4 3
PIN gat1/OUT1, gat2/IN1, gat3/IN1
NET net5 2
PIN gat2/OUT1, outbuffer1/IN1
NET net6 2
PIN outbuffer1/OUT1, outptSum
NET net7 2
PIN gat3/OUT1, gat5/IN1
NET net8 2
PIN gat4/OUT1, gat5/IN2
NET net9 2
PIN gat5/OUT1, outbuffer2/IN1
NET net10 2
PIN outbuffer2/OUT1, outptCarry
```

```
`timescale 1ns/1ps
module mel (N1,N2,N3,N6,N10);

input N1,N2,N3;
output N6,N10;
wire N4,N5,N7,N8,N9;

xor XOR2_1 (N4, N1, N2);
xor XOR2_2 (N5, N4, N3);
and AND2_1 (N7, N4, N3);
and AND2_2 (N8, N1, N2);
or OR2_1 (N9, N7, N8);
buf BUF1_1 (N6, N5);
buf BUF1_2 (N10, N9);

endmodule
```

d. 依照.v file 的格式完成輸出

```
`timescale 1ns/1ps
module mel (N1,N2,N3,N6,N10);

input N1,N2,N3;
output N6,N10;
wire N4,N5,N7,N8,N9;

xor XOR2_1 (N4, N1, N2);
xor XOR2_2 (N5, N4, N3);
and AND2_1 (N7, N4, N3);
and AND2_2 (N8, N1, N2);
or OR2_1 (N9, N7, N8);
buf BUF1_1 (N6, N5);
buf BUF1_2 (N10, N9);

endmodule
```

3. The Data Structure and Algorithm of My Program

a. net_vector: 用於紀錄 net 的資訊，包含每個 net 的編號、扮演的角色(input or output or wire)、以及對應到的 gate，一個 net 會用於多個 gate，故使用 vector 做紀錄

```
struct net_gate_relation{    //儲存gat5/IN1的對應關係
    int gate_id;             //幫gate編號方便net去對應
    string role;             //OUT1 or IN1 or IN2 or IN3
};

struct net_node{
    int net_id;
    string role;
    vector<struct net_gate_relation> relation_vector;
};
```

```
vector<struct net_node>net_vector;
```

b. gate_vector: 用於紀錄每個 gate 的資訊，包含每個 gate 的編號、gate 的種類(nand、or、and、xor、inv 等)、以及對應到的 net，因為一個 gate 至少會有一個輸入 net 與輸出 net，且該資料結構須被反覆查找，所以我們使用 vector 紀錄，如果有不只一個輸出 net 的時候，會用 vector.resize()增大 vector，紀錄後續的 net 編號。

```
struct gate_node{
    string gate_type;
    int gate_id;
    int gate_class_id;
    vector<int> gate_IO_vector{0,0};
};
```

```
vector<struct gate_node> gate_vector;
```

c. **output_object**: 用於紀錄輸出 file 會直接使用到的資訊，包含 input net、output net、wire net、以及記錄每個 gate 對應的 output net 與 input net 的 vector。**output_object 在寫入輸出檔的時候會被呼叫。**

```
class ouput_object {
    public: //在main中也可以被access
        vector<int> input_net_id;
        vector<int> output_net_id;
        vector<int> wire_net_id;
        vector<struct gate_node> gate_vector; //紀錄.v 的gate level描述，包含gate的種類、編號以及

        ouput_object(vector<struct net_node>net_vector,vector<string> gate_line){
            //建構input_net_id、output_net_id、wire_net_id
            for(int i=0;i<net_vector.size();i++){
                if(net_vector[i].role=="input")input_net_id.push_back(net_vector[i].net_id);
                else if(net_vector[i].role=="output")output_net_id.push_back(net_vector[i].net_id);
                else wire_net_id.push_back(net_vector[i].net_id);
            }
            sort(input_net_id.begin(),input_net_id.end());
            sort(output_net_id.begin(),output_net_id.end());
            sort(wire_net_id.begin(), wire_net_id.end());
            //建構vector<struct gate_node> gate_vector
            construct_gate_vector(gate_vector,net_vector,gate_line);
        }
};
```

4. Computation Steps of My Program

a. **read_file( )**:用於取輸入檔並存取 gate、net 和 pin 的輸入行

```
void read_file(string inputFilename, string& first_line, vector<string>& gate_line, vector<string>& net_line, vector<string>& pin_line){
    ifstream inputFile(inputFilename);
    if (!inputFile) {
        cerr<<"can't open input file: "<<inputFilename<<endl;
        return;
    }

    string line;
    bool found_first_line = false;

    while (getline(inputFile, line)) {
        string trimmed_line = trim(line);

        if (!found_first_line) {
            first_line = line;
            found_first_line = true;
        }
        else {
            if (!trimmed_line.empty()){            // 如果把空格都移除後不為空
                if (line.find("Inst") == 0) {
                    gate_line.push_back(line);
                } else if (line.find("PIN") == 0) {
                    pin_line.push_back(line);
                } else if (line.find("NET") == 0) {
                    net_line.push_back(line);
                }
            }
        }
    }
    inputFile.close();
}
```

b. **construct_gate_name_to_id_hash_table( )**:建構將 gate 的名稱對應到 gate id 的 hash table，用於建構 net_vector 中 net 對應 gate_id 的 relation vector

c. **output_object_PA1( )**:以 net_vector 中 net 對應 gate_id 的 relation 建構 gate_vector，確認每個 gate 的 OUT1 net 以及 IN1、IN2 net，如果某個 gate 有不只一個輸入 Net，會以 resize( )以及 push_back( )的方式，擴充 vector 以正確紀錄該 Gate 擁有的所有 Net。

d. **construct_net_vector( ):依照讀檔時存取的 net 資訊儲存每個 net 的資訊，尤其是每個 net 和 gate 的對應關係。**

e. **construct_gate_vector( ):依照讀 net_vector 儲存每個 net 的資訊去建構每個 gate 擁有的 net(包含輸入與輸出)，並記錄每個 gate 的類型以利輸出時使用。**

f. **write_file( )**:依照 output_object_PA1 寫檔

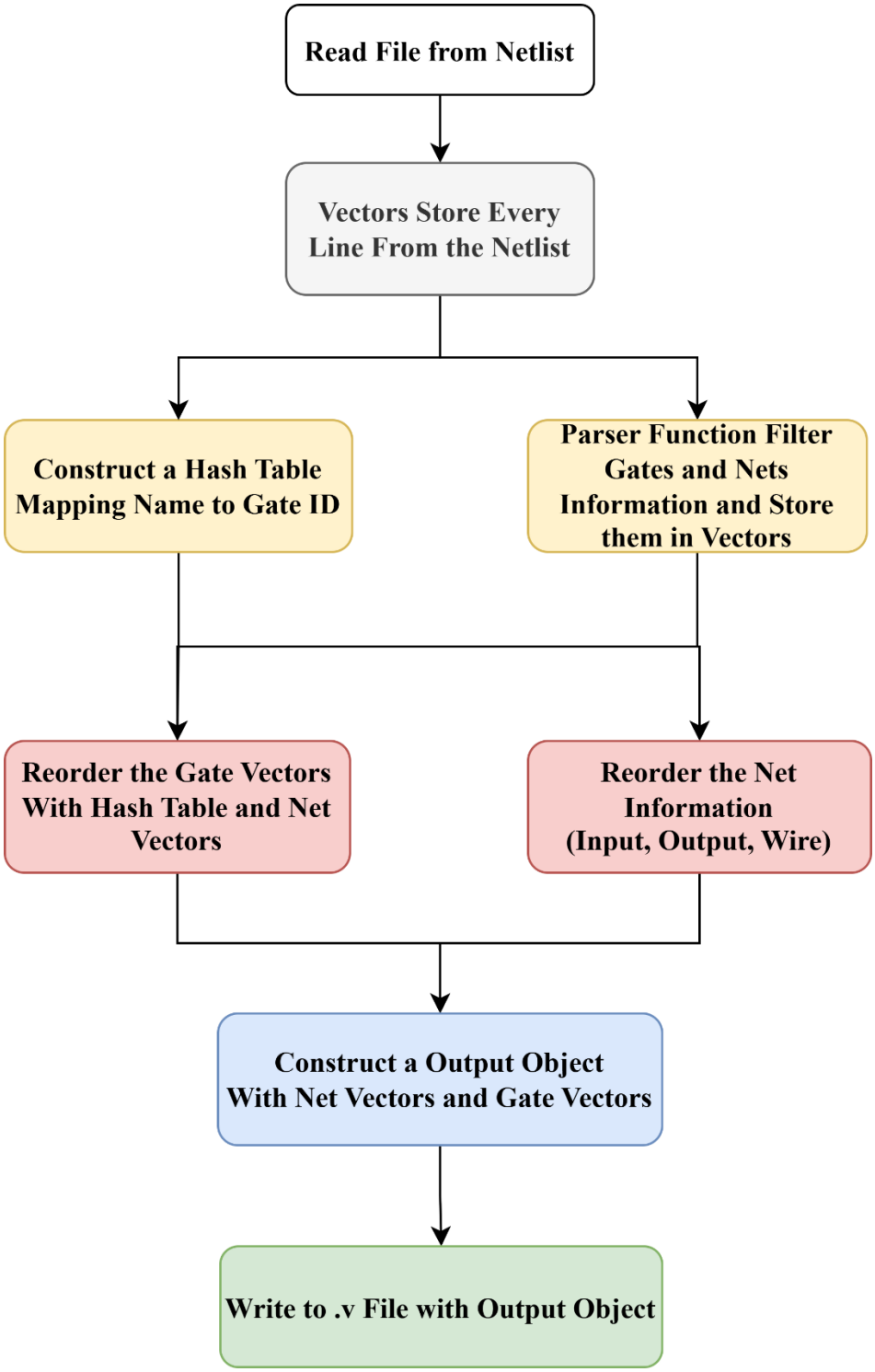
g. **PA1( ): 所有的 Function 會被 PA1( )所呼叫，而 PA1( )只接收輸出檔名以及輸入檔名的字串，保持 main( )的整潔，PA1( )如下:**

```
void PA1(string inputFilename,string outputFilename){
    string first_line;              //儲存input.txt line
    vector<string> gate_line;       //儲存input.txt line
    vector<string> net_line;        //儲存input.txt line
    vector<string> pin_line;        //儲存input.txt line

    unordered_map<string,int> gate_name_to_id_hash_table;
    //開一個hash table去統計所有gate對應的gate_id，construct_net_vector()會用到
    vector<struct net_node>net_vector;
    read_file(inputFilename,first_line,gate_line,net_line,pin_line);

    construct_gate_name_to_id_hash_table(gate_line,gate_name_to_id_hash_table);
    construct_net_vector(net_vector,net_line,pin_line,gate_name_to_id_hash_table);
    //建構net與gate和net的對應關係
    ouput_object ouput_object_PA1(net_vector, gate_line);
    write_file(outputFilename,ouput_object_PA1,first_line);
}
```

h. main( ):以 argc 和 argv 接收輸入檔與輸出檔的檔名，並呼叫 PA1( )以保持 main( )的整潔

```
int main(int argc, char *argv[]) {
    if (argc != 3) {
        cerr << "Input Command Wrong, use ./StudID_PA1.out <inputFilename> <outputFilename>\n";
        return 1;
    }
    string inputFilename = argv[1];     //讀取第一個參數作為輸入檔案名稱
    string outputFilename = argv[2];    //讀取第二個參數作為輸出檔案名稱
    PA1(inputFilename,outputFilename);
    return 0;
}
```

5. The Flow Chart of My Program



6. The hardness of this assignment and how you overcome it

a. 我覺得這次最難的應該是理解題目要求(例如.netlist 哪邊會有空格、會有多少空格、有躲多少斜線等)，要把題目看仔細才能知道規定.netlist 格式，才能設計對應的讀檔方式以及資料結構做正確的處理，**這邊真的感謝 PA1 的卓助教仔細回答我很多題目相關的問題!**

b. 這次作業需要將 gate 按照種類各自編號(for 輸出的.v file)，後來我是用 C++的 hash table (unordered map)做統計與對應，幫每個 gate 按照種類編號與計數，以利後續計算。

c. **如何判斷 output net、input net 與 wire net**:我本來是用.netlist 中 NET 那行的關鍵字來判斷 **output net、input net 與 wire net，後來發現助教沒有規定.netlist 的敘述中一定會出現關鍵字，所以後來我是這樣寫:**

```
if(OUT_count==0&&IN_count>0)my_net_node.role="input";
else if(OUT_count>0&&IN_count==0)my_net_node.role="output";
else my_net_node.role="wire";
```

我的寫法是如果該 NET 只有 OUT1，那必然是 **output net，如果**該 NET 只有 IN(例如 gat5/IN2、gat7/IN3 等)，那必然是 **input net，剩下的情況則是 wire net。**

7. Do you have any suggestions about this programming assignment

如果是建議的話，會比較希望 PA2 可以更早公布，因為 PA2 以後的難度相比 PA1 實在是難得許多，如果有更多時間可以準備會更好，而且通常 PA2 截止的時間已經接近期中考，時程上會忙碌許多(如果可以再期初就公布所有 PA 的話，我覺得會最好，因為學生在期初通常蠻有空的，但到期末就容易很多事情擠在一起，寫作業的時間也會大幅減少)。

這次真的很感謝 PA1 的卓助教仔細回答我這麼多關於題目設定的問題，真的很感謝助教百忙中抽空回答!祝助教後續的研究、論文與求職都順利!!!

8. Reference

a. **David Bryan, "The ISCAS '85 benchmark circuits and netlist format," available online:**

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=785838

b. **Collection of Digital Design Benchmarks, available online:**

https://ddd.fit.cvut.cz/www/prj/Benchmarks/index.php?page=download

c. **Cadence® NC-Verilog® Simulator Help, available online:**

https://picture.iczhiku.com/resource/eetop/WYKeftERsaRjPmcm.pdf

d. **Design Compiler - Synopsys**

https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html